第三章 运算符与表达式

- 基本运算符
- 表达式
- 类型转换

第三章 运算符与表达式

3.1 基本运算符 p28

运算符也称为操作符，是对程序中的数据进行运算的标志符号，参与运算的数据称为操作数或运算对象。

- 不同的运算符，需要指定的操作数的个数并不相同。根据运算符需要的操作数的个数，可将其分为三种：单目运算符（一个操作数）、双目运算符（两个操作数）和三目运算符（三个操作数）。
- 当不同的运算符混合运算时，运算顺序是根据运算符的优先级而定的，优先级高的运算符先运算，优先级低的运算符后运算。
- 在一个表达式中，如果各运算符有相同的优先级，运算顺序是从左向右，还是从右向左，是由运算符的结合性确定的。

第三章 运算符与表达式

分类	运算符
1、算术运算符	+ - * / %
2、关系运算符	> < == >= <= !=
3、逻辑运算符	! && ||
4、位运算符	<< >> ~ | ^ &
5、赋值运算符	=及其扩展赋值运算符
6、条件运算符	? :
7、逗号运算符	,
8、指针运算符	* &
9、求字节数运算符	sizeof
10、求类型运算符	typeid
11、强制类型转换运算符	(类型）
12、分量运算符	. →
13、下标运算符	[ ]
14、其他	如函数调用运算符（）

第三章 运算符与表达式

3.1.1 算术运算符

+ 、- （正号、负号，单目运算）

+ 、 - 、 *、 /、 %（双目运算）

+、-、*、/ 按通常的算术规定。

- 优先级：

  +、-（正号、负号）>

  *、/、% > +、-（加号、减号）

- 在搞不清优先级关系时，最好用括号‘（’、‘）’进行分离组合。

第三章 运算符与表达式

注意：

1. 对于除，若两个操作数都是整数，结果为整数（商部分）。

   1/2=0　　5/2=2

2. %通常称为取模运算，两个操作数必须都是整型数，结果为余数，余数的符号与左边数的符号等同。右边数为0时，发生严重错误，程序终止。

   3%2=1　　-3%2=-1　　3%-2=1　　-3%-2=-1

   8%4=0

3. 算术运算的结果可能太大，而不能存储在一个指定的变量中，这种情形称之为溢出。

   unsigned char k = 10 * 92; / / 溢出: 920 > 255

溢出举例

```
using namespace std;
#include <iostream>
int main()
{
        int a,b;
        a=32767;
        b=a+1;
        cout<<a<<'\n'<<b;
        return 0;
}
```

改为2147483647

运行结果

```
32767
32768
```

运行结果

```
2147483647
-2147483648
```

单选题 1分

设置

设有语句：int m=13, n=3, 则执行m %= n + 2 后，n 的值是

A 5

B 1

C 3

D 0

提交

第三章 运算符与表达式

3.1.8 自增、自减运算符 ++　　- -

```
i=6;
i++ ;      // i=i+1    i=7          ++i ;    // i=i+1    i=7
i- -;      //  i=i-1   i=6          - -i;    // i=i-1    i=6
```

```
int   i, j;
i=3;
j = ++i;  //前缀
i=4    j=4  先运算，后赋值
```

```
int   i, j;
i=3;
j = i++;  //后缀
i=4     j=3  先赋值，后运算
```

第三章 运算符与表达式

注意：

1. 自增、自减运算符只能用于变量，不可用于常量和表达式

   3++　　（x+y)++　　(-i)++

2. 优先级非常高

   -i++ ⟹ -(i++)

   i+++j ⟹ (i++) +j

3. ++和- -的结合方向为自右至左。

   ++i++ ⟹ ++(i++)　　✗

- 自增、自减的操作数必须是左值
- 前缀的表达式是操作数修改后的值，是左值
- 后缀的表达式是原先操作数的值，是右值

主观题 1分

设置

以下程序输出结果是？

```
#include <iostream>
using namespace std;
int main(void){
    int a=1,b;
    b=(a++)+(a++)+(a++);
    cout<<a<<'\t'<<b<<endl;
    return 0;
}
```

正常使用主观题需2.0以上版本雨课堂

作答

主观题 1分

设置

以下程序输出结果是？

```
#include <iostream>
using namespace std;
int main(void){
    int a=1,b;
    b=(++a)+(++a)+(++a);
    cout<<a<<'\t'<<b<<endl;
    return 0;
}
```

正常使用主观题需2.0以上版本雨课堂

作答

投票 最多可选1项

设置

编程练习-温度转换

- 题目内容：

输入一个摄氏温度C，输出对应的华氏温度F和绝对温度K（F=9/5*C+32，K=C+273.15），控制小数点后2位，之间用一个空格分隔

- 输入样例：

-3

- 输出样例：

F=26.60 K=270.15

A 通过测试

B 未通过测试

提交

第三章 运算符与表达式

3.1.2 关系运算符　　<、<=、>、>=、==、!=

- = = 与 = ： a=5;　赋值运算　　a= =5；判断是否相等
- a<=x<=b　与　数学上的x取值区间a≤x ≤ b　不同
- 优先级：

  算术运算符　<、>、<=、>=　= =、!=

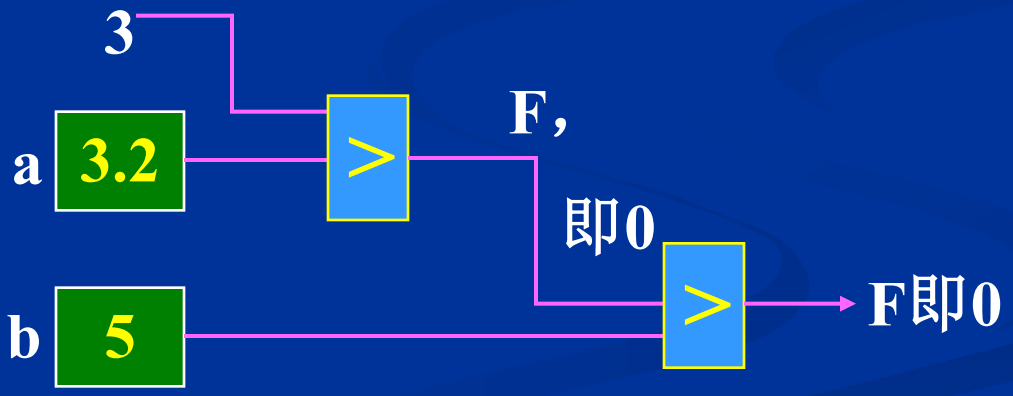
  高－－－－－－－－－－－－－－－－→低
- 运算结果是false（0）或true（1），C/C++中规定：

  C/C++中：用0代表假，非0代表真（许多情况下为1）

第三章 运算符与表达式

- 实例：
  设有定义 float a=3.2;int b=5;则：
  3>a>b //结果是?

根据右结合原则其运算过程如下：



第三章 运算符与表达式

- 实例：
  设有 3.3333+4.4444==7.7777
  结果：　　true　or　false ？

  ➢浮点数在计算机内进行运算和存储时会发生误差
  ➢当判断两个实数是否相等时，应转换成判断两个实数之差的绝对值是否小于一个给定的允许误差值　fabs(a-b)<=1e-6

单选题 1分

设置

下列程序的运行结果是

```
int main(void)
{   int a=5,b=4,c=3,d;
    d = (a > b > c);
    cout<<d<<endl;
   return 0;
}
```

A 5

B 3

C 1

D 0

提交

第三章 运算符与表达式

3.1.3 逻辑运算符

运算符：　　与&&　　或||　　非!

&&

A	B	结果
F0	F0	F0
F0	T1	F0
T1	F0	F0
T1	T1	T1

有假则假，全真才真

||

A	B	结果
F0	F0	F0
F0	T1	T1
T1	F0	T1
T1	T1	T1

有真则真，全假才假

!

A	结果
F0	T1
T1	F0

是假则真，是真则假

第三章 运算符与表达式

注意：

1. 优先级：

   !→&&→||

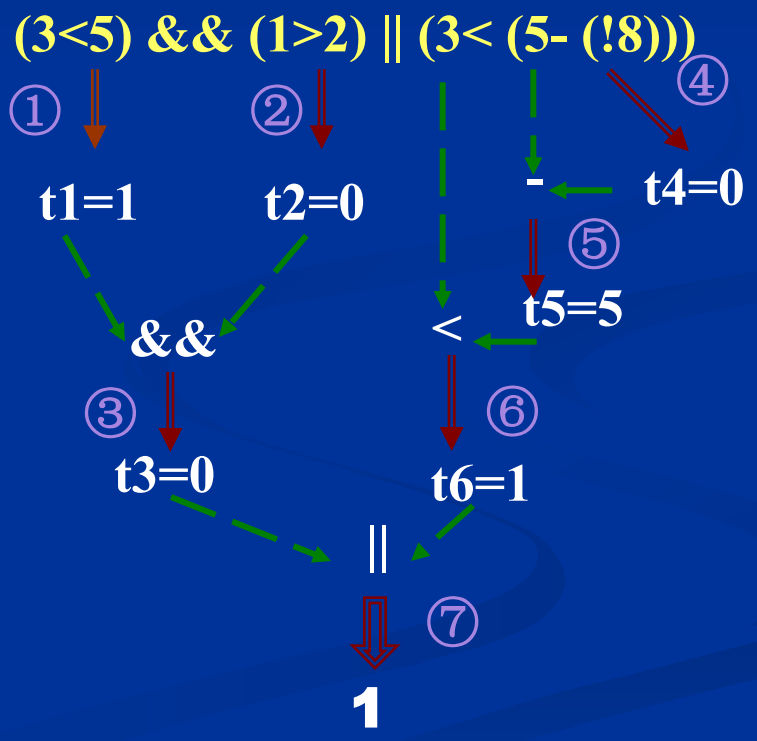
   ! →算术→关系→

   &&→|| →赋值→逗号

2. 作为条件，所有非0值均为真， 0为逻辑假；作为结果，逻辑真时，取值为1；否则取值为0；其运算结果可作为一个整数再参与逻辑运算、关系运算或算术运算 。

(3<5) && (1>2) || (3< (5- (!8)))

根据优先级与结合性，其求值顺序等同于：



第三章 运算符与表达式

注意（续）：

3. 不可写为 1<x<10　应为：1<x && x<10
4. 当前面的表达式可以得出整个表达式的结果时，不必再求后面的表达式，称为“求值优化”。
   - a&&b&&c　当a为0时，表达式为false，不必求b与c。
   - a||b||c　当a为非0时，表达式为true，不必求b与c。
   - 例：

     已知：int a=10, b=20, c=30;

     求：a<b || (c=2)　//结果以及a,b,c的值

     ① ↓

     true ⇒ 整个表达式的值为 true

     可见：表达式 c=2 已不需要计算

     所以：c不变仍为 30

第三章 运算符与表达式

例1：

判断某年是否为闰年

1）能被400整除　　year%400= =0

2）能被4整除，不能被100整除

year%4= =0&& year%100!=0

(year%400= =0) || (year%4= =0&&year%100!=0)

填空题 3分

设置

设有整型变量a、b、c,其中a,b的值分别是10与20，在执行语句 c=a++||b++后

a=［填空1］， b=［填空2］ ， c=［填空3］

正常使用填空题需3.0以上版本雨课堂

作答

第三章 运算符与表达式

3.1.4 位运算符

- 位操作运算符是用来进行二进制位运算的运算符。它分为两类：逻辑位运算符和移位运算符。
  1. 按位取反运算符“~”
  2. 左移运算符“<<”
  3. 右移运算符“>>”
  4. 按位与运算符“&”
  5. 按位或运算符“|”
  6. 按位异或运算符“^”

第三章 运算符与表达式

(1)移位运算符

①左移运算 <<

- 运算一般格式为：a<<n
- 其中a为整数，n为一个正整数常数。语义为将a的二进制数依次向左移动n个二进制位，并在低位补0。左移n位相当于乘以2的n次方，但移位运算的速度比乘法快。

②右移运算 >>

- 与左移运算符类同，将左操作数向右移动右操作数指定的二进制位数，忽略移位后的小数部分，并在高位补0。一个整数右移n位相当于除以2的n次方，但比除法快。
- 在VC++6.0中有符号数右移时高位补符号位，严格对应除法。

- 以上两种运算不影响数据本身的值，而是只产生一个中间量，这个中间量被引用后即不再存在。

第三章 运算符与表达式

(2)逻辑位运算符

①按位求反~

- 作用是将各个二进制位由1变0，由0变1。
- x=~a; 计算后变量a的值不变

②按位与运算 &

- 两个都为1时结果为1，否则为0。
- 例如：

```
a        01001101
b        00001111
a & b    00001101
```

- 该运算可用来将整数的某些位置0，而保留所需要的位，上例保留了低四位。

第三章 运算符与表达式

③按位或运算|

- 两个其中有一个为1则结果为1，两个均为0则结果为0。
- 例如：

```
  a       01001101
  b       00001111
 ---------------------
a | b     01001111
```

- 该运算符可用来将整数的某些位置1。上例高四位不变，低四位全1。

④按位异或运算^

- 两个不同则结果为1，否则为0。
- 例如：

```
  a       01001101
  b       00001111
 ---------------------
a ^ b     01000010
```

- 该运算符可用来将一个整数的某些位取反，或将整型变量的值置0（将整型变量与自身按位异或）。上例低四位取反，高四位不变。

单选题 1分

设置

a=-3&0xf运算后a的值是:

A -13

B 13

C -3

D 3

提交

填空题 1分

设置

a=~3运算后a的值是：［填空1］

正常使用填空题需3.0以上版本雨课堂

作答

填空题 1分

设置

a=5^2运算后的a的值是［填空1］：

正常使用填空题需3.0以上版本雨课堂

作答

投票 最多可选1项

设置

编程练习-低位置0

- 题目内容：

从键盘上输入一个整数x，将其低六位全部置0，其余各位不变。然后用十进制和十六制分别输出x的值，之间用一个空格分隔

- 输入样例：

100

- 输出样例：

127 7f

A 通过测试

B 未通过测试

提交

第三章 运算符与表达式

3.1.5 条件运算符

格式：<表达式 1 >？<表达式 2 >：<表达式 3 >

功能：先求出表达式 1 的值，若其值不等于 0 时，则求出表达式2的值,否则求出表达式3的值。所以作为运算的结果，或为表达式 2 ，或为表达式 3

例： a>=b?a:b;

第三章 运算符与表达式

注意：

- 条件表达式有一个结果值，通常和赋值结合。

  max=a>b?a:b;

- 条件表达式的优先级较低，仅高于赋值和逗号表达式。

  max=((a>b)?a:b) 等效于 max=a>b?a:b

- 为增加程序可读性，程序员习惯为<表达式 1 >加一对括号（）尽管不是必需的。

  max= (a>b)?a:b;

- <表达式 2 >和<表达式 3 >可为任何合法的C/C++表达式（可嵌套使用）。

  max=(a>=b?a:b)<c?c:(a>=b?a:b);

  max=(t=a>=b?a:b)<c?c:t;

第三章 运算符与表达式

3.1.6 赋值运算符

<变量>　＝　<表达式>

赋值运算符的优先级比算术运算符、关系运算符和逻辑运算符的优先级低，其结合性为从右向左。

a=b=5　⟶　b=5　a=5

第三章 运算符与表达式

- C++将变量名代表的单元称为“左值”，而将变量的值称为“右值”。左值必须是内存中一个可以访问且可以合法修改的对象，因此只能是变量名，而不能是常量或表达式。
- 例如下面的赋值运算是错误的：
  - 3.1415926=pi; //左值不能是常数
  - x+y=z; //左值不能是表达式
  - const int N=30;
    N=40; //左值不能是常变量

第三章 运算符与表达式

复合的赋值运算

a+=3 ⟹ a=a+3

x*=y+3 ⟹ x=x*(y+3)

x/=x-4 ⟹ x=x/(x-4)

x+=y ⟹ x=x+y

i+=j-- ⟹ i=i+(j--)

填空题 1分

设置

执行

```
int a=10;
a+=a-=a*a;
```

后，a的值为：[填空1]

正常使用填空题需3.0以上版本雨课堂

作答

第三章 运算符与表达式

3.1.7 逗号运算

表达式1，表达式2，表达式3，…，表达式n

① 逗号表达式的求解过程为自左至右，依次计算各表达式的值，最后一个表达式的值即为整个逗号表达式的值；

② 逗号运算符的优先级最低。

③ 使用逗号表达式的目的通常是想分别得到各个表达式的值，而并非一定要得到整个表达式的值。

④ 常用于for循环语句中，除此以外很少使用。

a=(3+4, 5*6, 2+1); ⟹ a:3

(a=3*5, a*4), a+5 ⟹ 表达式:20 a:15

(a=3*5, a)=5 ⟹ a: 5

第三章 运算符与表达式

例：下列语句中表达式中i, j的值各为多少

1、int i=0, j=0;
i=3, (j++)+i ;
i=3
j=1

2、 int i=0, j=1;
i+=j*=3;
i=3
j=3

3、int i=1, j=0;
j=i=((i=3)*2);
i=6
j=6

4、int i=1, j=1;
i+=j+=2;
i=4
j=3

第三章 运算符与表达式

3.1.9 sizeof运算符

单目运算符，用于求给出的类型、变量或常量在系统中存储时所需要的字节数

int a,b;

sizeof(a) ——存储变量a所需要的字节数

sizeof(a+b) ——存储a+b计算后的结果所需要的字节数

sizeof(int) ——存储int型数据系统所需要的字节数

sizeof(“a”) ——存储字符串常量 “a”所需要的字节数

sizeof(‘a’) ——存储字符a所需要的字节数

(1<<(sizeof(int)*8-1))-1 ——整型所能表示的最大整数

第三章 运算符与表达式

3.1.10 typeid运算符

单目运算符，用于获取表达式在运行时刻的类型信息。使用时要求包含<typeinfo.h>文件或标准库typeinfo文件，且需调用函数name。

typeid(表达式).name( )

int a=1; unsigned int b=2;

typeid(a+b).name() —— unsigned int

基本类型typeid输出类型标识

类型名	VS typeid(expr).name()	GCC typeid(expr).name()
bool	bool	b
char	char	c
signed char	signed char	a
unsigned char	unsigned char	h
char8_t	char8_t	Du
wchar_t	wchar_t	W
char16_t	char16_t	Ds
char32_t	char32_t	Di
[signed] short [int]	short	s
unsigned short [int]	unsigned short	t
[signed] int	int	i
unsigned int	unsigned int	j
[signed] long [int]	long	l
unsigned long [int]	unsigned long	m
[signed]long long[int]	__int64	x
unsigned long long[int]	unsigned __int64	y
float	float	f
double	double	d
long double	long double	e
std::nullptr_t	std::nullptr_t	Dn
void	void	v

第三章 运算符与表达式

3.2 表达式

- 表达式是由运算符、括号和操作数构成的序列，运行时能计算出一个值的结果。
- 操作数可以是常量、变量和表达式。
- 表达式按其中运算符的优先级和结合性来求值。最终都有确定的运算结果和类型。

第三章 运算符与表达式

- 能放在赋值运算符左边的表达式为左值表达式，简称左值。左值表达式必须能指定一个存放数据的空间，通常为变量。
- 能放在赋值运算符右边的表达式为右值表达式，简称右值。
- 由算术运算符、位运算符和操作数构成的表达式为算术表达式。
- 由赋值运算符和操作数构成的表达式为赋值表达式。
- 由关系运算符和操作数构成的表达式为关系表达式。
- 由逻辑运算符和操作数构成的表达式为逻辑表达式。
- 由逗号运算符和操作数组成的表达式为逗号表达式。

第三章 运算符与表达式

左值表达式和右值表达式

- 能放在赋值号左边的表达式，称为左值表达式，简称左值。
  - ✓左值应指定一个能存放数据的空间，如变量。
  - ✓int a=3;　a是左值；　a+1不是左值。
  - ✓++,--必须作用于左值。
  - ✓const int b = 3; b是常量左值。
  - ✓++a是左值。但不要执行++a = a++;结果不统一！
- 能放在赋值号右边的表达式，为右值表达式，简称右值。
- 所有左值都可作为右值；但右值不一定能作为左值。

第三章 运算符与表达式

3.2.2 表达式语句

1. 任何一个表达式后加上分号“；”就构成了表达式语句。
2. 空语句仅由一个分号组成，不进行任何操作。

第三章 运算符与表达式

3.2.3 decltype

C++11 引入运算符decltype(表达式)，得到指定表达式的类型，可用于说明变量的类型。

例如：

```
int i;
decltype(i) j = 0; //等同于int j = 0 或 auto i = 0;

float a;
double b;
decltype(a + b) c; //等同于double c;
```

第三章 运算符与表达式

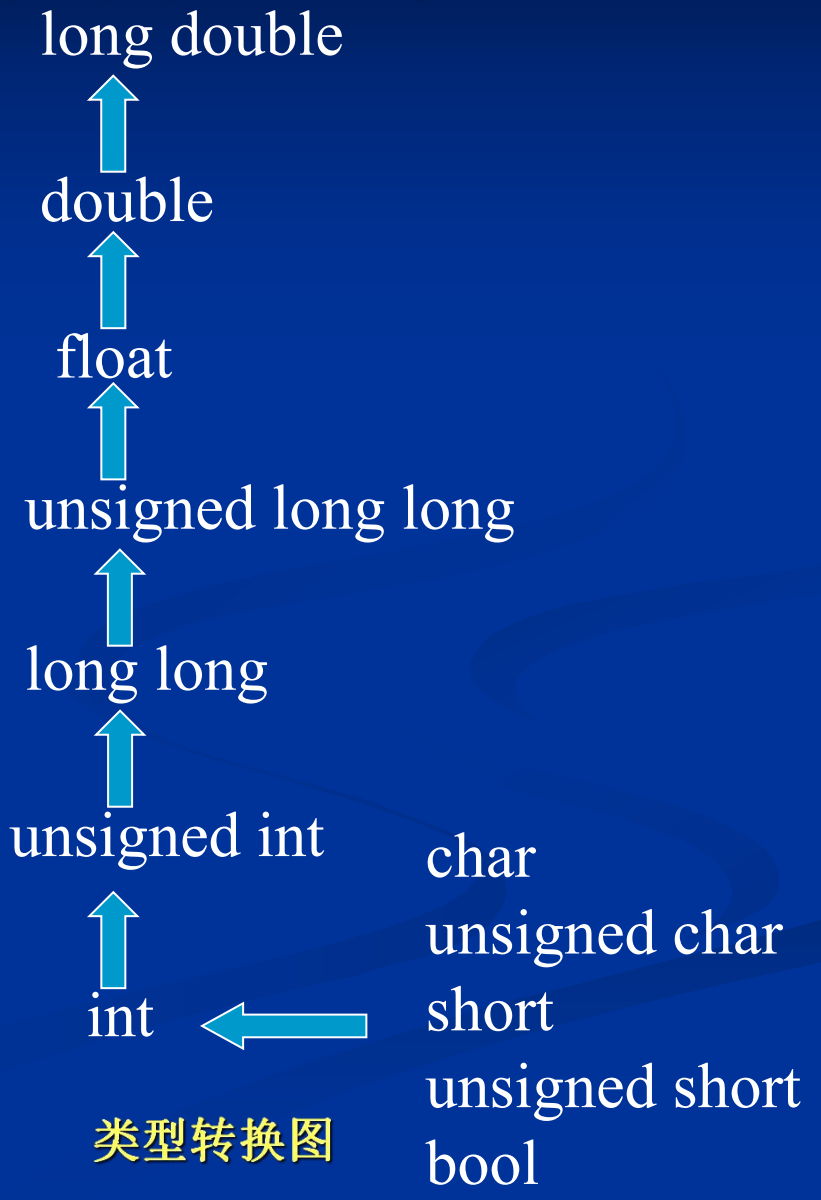
3.3 类型转换

(1)不同类型混合运算时自动类型转换

整型、实型、字符型数据间可以混合运算。在运算时，不同类型的数据要先转换成同一类型的数据再进行运算。

第三章 运算符与表达式

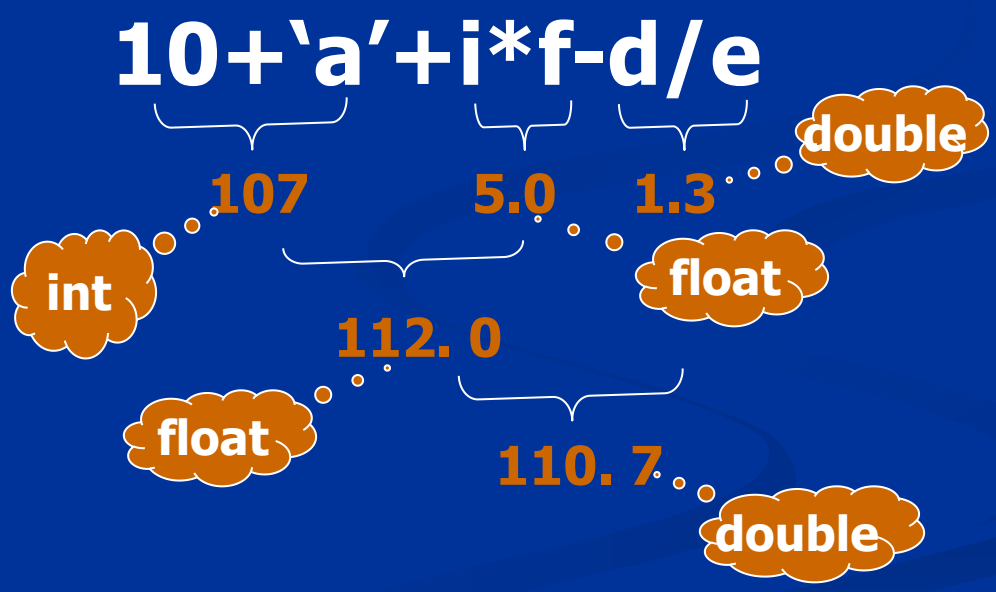
- 转换规则如下：
  - ① 操作数为布尔、字符或短整型时，系统自动变换成整型。
  - ② 其余情况，当两操作数类型不同时，将精度低（或表示范围小）的操作数的数据类型变换到与另一操作数类型相同再进行运算，即向高精度靠。
  - ③ unsigned int和long混合运算时，转成unsigned long。
  - ④ 任意两个值之间进行逻辑运算或关系运算，其结果都是一个布尔值。



类型转换图

思考：根据已知变量定义及赋值语句，计算以下表达式的值，并说明每一步所得结果的数据类型。

```
int  i;
float  f;
double  d;
long  e;
……
i=2;
f=2.5;
d=3.9;
e=3;
……
```



单选题 1分

设置

有下面语句： auto a = 2u; auto b = -3; auto c = a * b; c 的类型是

A unsigned int

B int

C long

D 语法错误

提交

单选题 1分

设置

有下面语句： auto a = 2u; auto b = 3l; int i = 1; auto v = i == 2 ? a : b;　　v的类型是

A unsigned int

B int

C long

D unsigned long

提交

第三章 运算符与表达式

(2)赋值时的类型转换

若" = " 两边变量类型不同，在赋值时要进行类型转换。

转换原则：根据左边变量的类型转换。

1）将实型数赋给整型变量时，去掉小数部分，仅取其整数部分赋给整型变量。

2）将整型数赋给实型变量时，将整数变换成实型数后，再赋给实型变量。

第三章 运算符与表达式

3) 字符型数据赋给整型变量，分二种情况：

- 对无符号型的字符变量，低八位不变，高位补 0 后赋值；
- 而对于有符号型的变量，若字节的符号位为0时，与无符号型的转换规则相同；当该字节的符号位为1时，将高位全部置1后再赋值。

4) 将实型数赋给整型、无符号（长）整型赋给整型变量（即高精度赋给低精度）时，当超出取值范围，赋值的结果错误。

5) 一个无符号整数和一个有符号相同类型整数之间相互转换时，将保持原补码值不变。

思考：以下程序的输出结果是什么？

```
using namespace std;
#include  <iostream>
int  main()
{
      unsigned int x=-1;
      cout<<"x="<<x;
      return 0;
}
```

程序运行结果：

```
x=4294967295
```

结论：有符号型向无符号型转换时，补码内容不变

思考：以下程序的输出结果是什么？

```
using namespace std;
#include  <iostream>
int  main()
{
      int c = -4;
      unsigned a = 2;
      cout << boolalpha << (c < a) << endl;
      return 0;
}
```

程序运行结果：

```
false
```

思考：以下程序的输出结果是什么？

```
using namespace std;
#include  <iostream>
int  main()
{
      char c = 250;
      int a = c;
      cout<<a<<endl;
      return 0;
}
```

程序运行结果：

```
-6
```

单选题 1分

设置

设有说明语句：double x, y; 则表达式x=3, y=x+5/3 的值是

A 4.66667

B 4

C 4.0

D 3

提交

单选题 1分

设置

设有语句：int a=13, b=9, c; 执行c = a / b + 0.8后，c 的值为

A 1.8

B 1

C 2.24444

D 2

提交

单选题 1分

设置

以下程序的输出结果为：

```
#include <iostream>
using namespace std;
int main() {
        int a=3;
        unsigned b=-1;
        unsigned long long c;
        c=a*b;
        cout<<c;
        return 0;
}
```

A -3

B 4294967293

C 12884901885

D 18446744073709551613

提交

第三章 运算符与表达式

(3)强制类型转换

通过强制类型转换可将某种类型的数据看成另一种类型数据

例：

```
int a=5,c=2;
double d;
d=a/c;  //d中为2
d=(double)a/c;  /d中为2.5，此时将a作为double型数据处理
```

在C++中，(double)a也可以写成 double(a),在C中只允许前一种写法

思考：以下程序的输出结果是什么？

```
using namespace std;
#include  <iostream>
int  main()
{
      float  x;
      int  i;
      x=3.6;
      i=(int)x;
      cout<<"x="<<x<<',';
      cout<<"i="<<i;
      return 0;
}
```

程序运行结果：

```
x=3.6,i=3
```

结论：较高类型向较低类型转换时可能发生精度损失问题

举例：已知三角形的边长a、b、c，求三角形周长的一半。

```
#include <iostream.h>
void main()
{
    float a,b,c,s;
    a=3;b=4;c=5;
    s=1/2 * (a+b+c);
    cout<<"s="<<s;
}
```

改为：1.0/2　　(1/2.0)
或　　(float)1/2　(1/(float)2)



程序运行结果：

```
s= 0
```

程序运行结果：

```
s= 6
```

投票 最多可选1项 设置

编程练习-总和与平均数

- 题目内容：

在键盘输入4个整数，均在int类型取值范围内，计算并输出这4个数的总和和平均值，之间用一个空格分隔

- 输入样例：

1 2 3 4

- 输出样例：

sum=10 average=2.5

A 通过测试

B 未通过测试

提交